PORTFOLIO

# Septian Saputra



*Selected work — QA, StafBook (2021–Present) · Product, StafBook (Jul 2026–Present)*

| 943 | 301 | 42 | 02:00 | 6 | 2 |
|---|---|---|---|---|---|
| API endpoints inventoried | automated tests committed | mobile flows (Maestro) | daily CI run, WIB | features verified in production | PRDs shipped |

***StafBook's first QA hire*** *— on board before the login screen existed, with comprehensive product knowledge across every module on both web admin and mobile ESS: scheduling, attendance, leave, payroll, reimbursement, cash advance, loans, and approval workflows.*

# 1 · API Regression & Production Verification Platform

*StafBook backend (api.sbook.dev) · Playwright + TypeScript · GitHub Actions*

StafBook's backend shipped with no real test safety net — ~37 unit tests on an in-memory database, and a deploy pipeline with no test stage. Regressions were only ever caught by manual QA. Starting July 2026, I designed and built an automated regression platform to close that gap — everything below, in about two months.

- Inventoried and prioritized **943 API endpoints** from the backend source, then built a tiered regression suite (smoke → P0 → P1) covering the highest-risk paths first.
- Engineered a **3-tier test-data architecture** — read-only, write-safe, and destructive — so regression could run safely against a shared staging environment without corrupting data other tests or manual QA depend on.
- Wired the suite into **CI running daily at 02:00 WIB** (GitHub Actions), with failures automatically filed to Jira and de-duplicated — the same defect never reopens a stale ticket.
- Surfaced **30+ candidate defects** through systematic backend code analysis before any test was run — reading route handlers, validation rules, and policies to find where "the status code looks fine" doesn't mean "the behavior is correct" (e.g. endpoints that always return 200 regardless of outcome, GET requests that mutate state).

**Production:** built and ran a separate, safety-gated post-deployment suite live on app.stafbook.com — hard guardrails (single dedicated QA business, no real fund movement, payment recording only via Manual Transfer, never Smart Disburse). Used to prove 6+ feature releases directly on production: payroll prorata, PPh21 tax withholding, and attendance changes.

# 2 · Payroll & Tax Compliance Automation

*Indonesian PPh21 / BPA1 / prorata domain · oracle-based verification*

Payroll and tax calculations are the highest-consequence area of the product — a wrong number is a wrong paycheck. I built verification that doesn't trust the system under test to grade itself.

- Wrote **independent calculation oracles** for Indonesian income tax (PPh21 TER rates and Pasal 17 progressive brackets) directly from the regulation text — not copied from the application's own code — so a shared bug in both places can't hide.
- Built the **Prorata "Hari Kerja Tetap" fixture**: multi-case staff across shift, schedule, attendance, and leave-policy combinations, including offboard + rehire inside a single payroll cycle — the scenario that exposed a real calculation bug (below).
- Designed a **single-source-of-truth reporting pipeline**: one JSON file generates a live web dashboard, a Google Sheet, and an 11-tab Excel archive. Pass/fail is computed by comparing recorded reality to expectation — never hand-typed — so the report can't claim a pass that didn't happen.

Live evidence: dashboard · sign-off record

(claude.ai/artifact/DmWwBi99aYzsfpSDUPvpDp · claude.ai/artifact/SVBsU5byLb4h3ie4w5Lga7)

# 3 · Notable Find — Overtime Payroll Miscalculation

*Cross-platform E2E (Web ↔ Mobile) · dual-channel approval*

Running a full cross-platform overtime scenario — policy creation on Web through approval on both Web and Mobile — surfaced a case the manual test plan hadn't covered: a shift submitted with identical start and end time.

- Expected: 0 minutes of overtime. Actual: the system computed and displayed 24 hours.
- Escalated as P0/blocker rather than a minor edge case, because the record was reachable through bulk-approve — a single click could pay a full day's overtime wage for zero minutes worked, with no way to reverse an approval once made.
- Documented with reproduction steps, expected vs. actual, and a root-cause hypothesis before handing to engineering.

## 4 · Product Requirements Shipped

*Client-sourced requirements → shipped acceptance criteria*

**Overtime Enhancement — Break Time, Policy Cleanup & Consistency** — triggered by a logistics client whose staff took breaks mid-shift that the system counted as full paid overtime. Scoped into 3 features and 10 requirements split across Web, Mobile ESS, and Backend, each with explicit acceptance criteria. Ran the requirement-clarification loop through to resolution on 7 open questions with the client via CX before implementation started.

**Face Recognition Attendance Mapping — Business-Level Optional & Staff-Level Mandatory** — replaced an ambiguous single toggle with two explicit enforcement modes (strict per-staff block vs. business-wide optional), closing a recurring support-ticket source. 4 user stories plus a 4-scenario configuration matrix covering every combination of policy and photo status.

## 5 · Process & Team Leadership

*Scrum → Scrumban · daily delivery rhythm · tooling*

- Run the team's daily delivery cadence — daily standup, sprint planning, and backlog grooming for the Product & Tech team — and facilitate SIT and UAT cycles before every release, coordinating sign-off between engineering, product, and client stakeholders.
- Led the Product & Tech team's transition from Scrum to Scrumban: designed and built an 8-column Jira workflow board (Backlog → Done) with WIP limits and swimlanes by work type (feature, bug, CX, infrastructure).
- Automated CX ticket intake with an Airtable ↔ Jira webhook, removing a manual copy step from every client-reported issue.
- Delivered weekly Product & Engineering updates to executive leadership, tailored to three formats — full deck, chat-length summary, and email.

Track record: **1,674 of the team's 4,574 Jira issues filed (36%)** — the highest volume of any teammate — bug reports, requirements, and QA tickets, sustained from manual QA days through today. Authored and signed off structured test cases (positive/negative, expected vs. actual) across **33 sprints of UAT and 12 sprints of SIT**.

# 6 · Independent AI-Orchestrated Projects

*Personal work · directing AI coding agents as the primary build method*

Outside StafBook, AI-agent orchestration is my daily working method for shipping software and content end-to-end — from architecture decisions down to verification. Two examples:

**Agentic Studio** — a 6-role AI agent orchestration system (Product Manager, UI/UX, Backend, Frontend, QA, Mobile) that plans, builds, and ships web features with a human owner giving intent, not code. Designed the coordination model myself: a shared task board as single source of truth, structured (schema-validated) output per role so agents hand off machine-readable work instead of free text, and a QA role that runs **real automated tests against a real running server** — pass/fail comes from process exit codes, not model judgment. Failed QA automatically files a scoped fix task back to the responsible role and re-tests, escalating to a human only after 3 failed attempts. Verified end-to-end before connecting a real model: full pipeline (Owner instruction → PM → backlog → UI/UX → Backend → Frontend → QA fail → auto-fix → QA pass → Mobile) traced clean with a stand-in for the API call.

**FoodInsideAI** — produced two data-driven digital ebooks (90 and 73 pages) end-to-end as paid digital products: directed research and fact-verification against regulatory standards (BPOM label-claim thresholds, TKPI 2017, USDA FoodData Central), with **160–350+ cited sources per title** and an explicit rule set for what claims the data does and doesn't support. Built the production pipeline myself — merge → QC audit → manuscript → slide deck → publish-ready PDF — as a repeatable, re-runnable process rather than one-off manual formatting.

| 2,666 | 1.75M | 1.5M | 23 |
|---|---|---|---|
| Instagram followers, @food.insideai | views in a 30-day period | views on a single top video | posts to reach that audience |

Growth was organic and fast: **2,666 followers in under 3 weeks**, with one video reaching **1M views in about 3 days**.

Content — image, video and voice — built with **Midjourney, ElevenLabs, and Flow AI**; copy and captions with ChatGPT and Claude. A second brand, **Hidden.Indo** (Instagram, TikTok, Facebook), applies the same production stack to breaking-news-style technology and finance content.